

**Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
«Высшая школа экономики»
Факультет компьютерных наук**

Дубровин Антон Романович

Реализация пайплайна загрузки данных

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ
по направлению подготовки 09.04.04 «Программная инженерия»
образовательная программа «Инженерия данных»

Рецензент

(ученая степень, ученое звание)

Буланцев Дмитрий Валерьевич

(И.О. Фамилия)

Руководитель

(ученая степень, ученое звание)

Бояр Владислав Денисович

(И.О. Фамилия)

Москва 2026

Содержание

Аннотация	3
Введение	4
1 Обзор существующих решений	6
2 Проектирование системы	11
2.1 Формальная постановка задачи	11
2.2 Описание предметной области и источников	12
2.3 Архитектура решения	13
2.4 Обоснование выбора технологий	14
2.5 Модель данных	15
2.6 Описание MOEX ISS API	18
2.7 Выводы по главе	21
3 Реализация	22
3.1 Адаптеры источников данных	22
3.2 Оркестрация и DAG-и	24
3.3 Стратегии записи в PostgreSQL и ClickHouse	25
3.4 Контроль качества данных	27
3.5 Витрины данных и дашборды	29
3.6 Telegram-бот для взаимодействия с системой	32
3.7 Инфраструктура и Docker-архитектура	34
3.8 Выводы по главе	34
4 Тестирование и экспериментальная оценка	35
4.1 Функциональное тестирование и эксплуатационные характеристики	35
4.2 Эксперимент PostgreSQL vs ClickHouse	36
4.3 Выводы по главе	37
Заключение	39
Список использованных источников	40

Аннотация

В рамках магистерской дипломной работы по направлению «Инженерия данных» был разработан пайплайн загрузки данных, покрывающий полный цикл процесса работы с данными. Пайплайн включает в себя загрузку данных, обработку и трансформацию данных, оркестрацию процессов и работу с данными на примере различных сценариев обработки биржевых данных.

В рамках реализации источником данных была выбрана Московская биржа (MOEX), а именно работа с информацией по индексам, акциям, дивидендам и др. Архитектура системы разработана с возможностью добавления и других источников данных, например другие биржи, новостные каналы и информационные сервисы.

Цель проекта - создание комплексного сервиса для анализа биржевых данных, который покрывает собой полный цикл работы по анализу данных и использует популярные подходы, такие как ETL, оркестрация, разделение баз данных для быстрого и аналитического хранения.

В основе решения лежит оркестрация процессов с помощью Apache Airflow, в рамках которой под каждую отдельную сущность реализован DAG, включающий в себя последовательные операции из полного цикла по выгрузке, обработке и загрузке данных в БД.

Сбор данных осуществляется двумя различными способами. Первый включает в себя как получение исторических данных по сущности за определенный промежуток времени, так и получение общих данных (справочники и т.д.). Второй - получение данных в реальном времени с помощью WebSocket.

Хранение данных осуществляется в двух отдельных БД: PostgreSQL (OLTP) для хранения актуальных данных по каждой сущности, и ClickHouse (OLAP) для хранения исторических данных. По данным из СН строятся аналитические витрины.

Данные об актуальных ценах из PostgreSQL и исторические данные из аналитических витрин используются для построения дашбордов в Grafana.

В качестве интерфейса для пользователя был разработан Telegram-бот с функционалом по триггеру DAG Airflow, поиску сущностей в БД, и просмотру текущей информации по ним.

Ключевые слова: ETL, MOEX, API, Apache Airflow, DAG, PostgreSQL, ClickHouse, REST, WebSocket, биржа, пайплайн, OLAP, OLTP.

Введение

В настоящее время популярность инвестиций, количество частных инвесторов и совершенных на биржах сделок стабильно растет [1] [2], в связи с чем растет и спрос на данные для анализа рынка. На текущий момент существуют различные варианты, покрывающие лишь часть потребностей, но комплексного открытого решения нет.

Для построения такого решения в качестве источника данных была выбрана крупнейшая биржевая площадка в России, а именно Московская биржа MOEX. Данный выбор обоснован следующими причинами:

- **Открытые данные.** Большую часть информации можно получить бесплатно и без брокерского договора;
- **Актуальность и количество данных.** Московская биржа представляет более 600 компаний, обслуживает более 40 млн частных инвесторов, 76 млн счетов, оборот 2,5 трлн рублей [2];
- **Насыщенность и полезность данных.** По данным Московской биржи можно получить всю необходимую информацию для хранения и анализа: как исторические, так и актуальные цены акций, облигаций, индексы, дивиденды и др.

Однако, MOEX предоставляет пользователю только возможность получения данных по API. Обработка и аналитика на стороне биржи не предусмотрены.

В связи с этим для данной работы была поставлена конкретная задача - разработать систему, покрывающую полный процесс анализа и работы с данными. Система должна включать в себя получение данных из Московской биржи, локальное хранение данных и работу с ними.

Предлагаемая система собирает данные из MOEX двумя способами, а именно **REST API** [5] - для получения исторических данных за конкретные промежутки времени, справочных данных, или данных, обновляемых с низкой периодичностью и **WebSocket** - для получения данных в реальном времени, и хранит эти данные в двух базах данных: **PostgreSQL** - реализована как OLTP-хранилище для актуальных данных по сущностям и **ClickHouse** - реализована как OLAP-хранилище для хранения исторических данных.

Для оркестрации процессов используется **Apache Airflow**. Для каждой сущности, например, истории индексов, всех инструментов из биржи, корпоративных событий (дивиденды) и др. создаётся отдельный **DAG**, который включает в себя получение соответствующих данных MOEX по REST API, преобразование в модель данных и загрузку в базы данных PostgreSQL и ClickHouse.

Разрабатываемая система представляет собой не столько классическую **ETL**-систему, сколько **ETLT**-систему. Архитектура организована таким образом, что реализует не только обычные шаги данного процесса (получение данных из MOEX, предварительное преобразование, загрузка в БД), но и позволяет произвести дополнительный этап трансформации по обработке данных, получаемых уже из локальной БД (например, для витрин данных), а также интеллектуальный или глубокий анализ (например данных, получаемых из новостных лент).

По данным из ClickHouse строятся аналитические витрины - view внутри БД. Представляют собой SQL-запросы для выборки определенных сырых данных в готовом для отображения в

дашбордах виде.

Для отображения данных пользователю используются дашборды в Grafana. Подключение происходит к обеим БД - из ClickHouse отображается информация из аналитических витрин, из PostgreSQL - текущая информация по инструментам и индексам.

Помимо дашбордов, реализован Telegram-бот, который предоставляет интерфейс пользователю в удобном виде. В возможности бота входит триггера DAG Airflow для загрузки любой необходимой информации, поиск сущностей в БД и выдача текущего состояния инструментов и индексов.

Работа имеет высокую практическую ценность - система работает с реальными биржевыми данными и является базой для масштабирования аналитическими модулями. При этом работа затрагивает все основные направления инженерии данных - построение ETL пайплайна, оркестрация процессов, работа с OLTP и OLAP хранилищами, сбор исторических данных и сбор в реальном времени, контейнеризация, контроль качества данных.

В данной дипломной работе в главе 1 рассматриваются существующие системы, работающие с биржевыми данными, описываются их отличия от предлагаемого решения, их недостатки, раскрываются преимущества разрабатываемой системы перед ними.

Вторая глава данной работы посвящена проектированию системы: построению архитектуры, выбору технологий для каждого этапа этой архитектуры. Также при проектировании исследовалось API Московской биржи, существующие сущности, методы и возвращаемые данные и их структура. На основе этого была построена модель для данных, выгружаемых из биржи и преобразуемых для дальнейшего хранения и работы в рамках системы.

В третьей главе представляется реализация построенной системы, а именно реализация ETL-процесса для загрузки данных из биржи, их преобразования и записи в БД с учетом особенностей хранения данных в двух базах PostgreSQL (OLTP) и ClickHouse (OLAP), и контроля загруженных данных. Рассматривается реализация оркестрации процессов с помощью Airflow, построение витрин и дашбордов, а также telegram-бот для взаимодействия с системой. В последней секции данной главы рассматривается построение общей инфраструктуры, и Docker-архитектура решения.

Четвертая глава посвящена тестированию и сравнению производительности PostgreSQL и ClickHouse, приведены метрики качества данных.

В заключении подведены итоги работы, рассмотрены положительные и отрицательные результаты по итогам разработки данной системы, рассмотрены перспективы дальнейшего развития.

1 Обзор существующих решений

В данной главе представлены существующие инструменты для работы с биржевыми данными, приведено описание этих решений, их возможности и ограничения. Приведено сравнение с предлагаемой системой по различным критериям, например: автоматизация сбора данных с биржи, хранение и доступность данных, скорость доступа к данным, аналитика, зависимость от внешних сервисов, отсутствие платной подписки и регистрации.

Первым рассматриваемым существующим аналогом является **AlgoPack**. Данное решение является официальным сервисом Московской биржи [4] для предоставления информации по биржевым данным. У этого сервиса также существует обертка над API для Python - библиотека **moealgo** [3].

В вопросе получения данных из MOEX AlgoPack предлагает широкий функционал, а именно получение как всех исторических данных начиная с 2020 года, так и данных в реальном времени - свечи, стаканы котировок, информация о торгах и т.д. С помощью библиотеки **moealgo** можно получать данные в виде **pandas DataFrame** или **json**, что будет удобно для дальнейшей работы с данными.

Однако, преимущества AlgoPack+moealgo на этом заканчиваются. Во-первых, потому что решение является исключительно системой для получения данных с биржи. Отсутствует хранение данных - каждый запрос идет напрямую к API MOEX, получает и отдает данные. При необходимости повторного получения данных необходимо заново запрашивать их по запросу к API. Для возможностей аналитики это имеет конкретные недостатки - отсутствие быстрого доступа к данным. А в случае получения исторических данных за длительный период, появится необходимость использовать **batch** запросы из-за ограничений ответов API на 100/500 записей.

Во-вторых, данный аналог не находится в открытом доступе. Для начала работы с ним требуется пройти регистрацию, а для полного доступа - необходима платная подписка.

Далее будут описаны основные недостатки данного решения:

- Отсутствует хранение данных;
- Нет автоматизации - все данные необходимо каждый раз запрашивать вручную;
- Расписания и мониторинг необходимо настраивать все самостоятельно;
- Нет трансформации данных - библиотека отдает сырые данные напрямую с биржи;
- Нет аналитики данных - запросы в БД, **view**, витрины, дашборды;
- Скорость получения данных - из-за отсутствия хранения данных приходится каждый раз обращаться к API, особенно критично в случае исторического запроса за длительное время, из-за получения данных по **batch**'ам;
- Отсутствие открытого доступа - нужна регистрация, а для полного доступа - платная подписка.

Вывод по Algorack+moexalgo: система решает исключительно задачу извлечения (Extract) из процесса ETL, но не обеспечивает работоспособность полного цикла процесса работы с данными.

Следующий рассматриваемый аналог - **Finam Export** - популярный инструмент для работы с историческими данными биржи, разработанный брокером Finam.

Finam Export позволяет работать с историческими данными, причем поддерживает не только MOEX, но и другие биржи. Также предоставляет возможность ручной выгрузки информации в файл.

Недостатки:

- Основной режим работы данного инструмента - ручной. Хотя и существуют сторонние различные решения, например библиотека `finam-export`[6], которые предоставляют возможность автоматизировать выгрузку - они не являются официальными;
- Отсутствует получение данных в реальном времени - только исторические данные;
- Аналогично Algorack - нет хранения данных, расписания, мониторинга, трансформации и аналитики данных, медленная скорость получения данных.

Вывод по Finam Export следующий: инструмент подходит скорее для разовых ручных работ по историческим данным, но не имеет возможности для построения автоматизированных процессов по анализу данных.

Trading View [7] является одной из самых популярных веб-платформ для визуализации и анализа финансовых данных.

Trading View отличается от ранее представленных аналогов и является платформой для просмотра данных в онлайн-формате. Поддерживает как MOEX биржу, так и остальные, настройку интерактивных графиков с отображением финансовых данных, настройку алертов. Также платформа поддерживает социальную активность инвесторов, так как доступна возможность делиться графиками между собой. Кроме того, разработчики платформы создали свой оптимизированный для работы с рыночными данными язык программирования Pine Script [8], который реализует возможности по созданию индикаторов и торговых стратегий.

Платформа имеет ряд ограничений:

- Все данные доступны только внутри платформы. Их нельзя получить или выгрузить вовне, они доступны только для ручной работы и только внутри Trading View;
- Несмотря на наличие удобного инструмента в виде языка Pine Script, он также доступен только внутри платформы, и написанные в нем скрипты нельзя использовать для связи в БД, для своих внешних разработок и т.д.;
- Внутри решения в рамках бесплатной подписки доступна малая часть возможностей. Для доступа ко всем возможностям необходима платная подписка.

Вывод: Trading View является сильным инструментом для ручной работы и анализа инвесторов, но не подходит для любых внешних доработок и расширений для построения обширных

процессов, так как все возможности и данные доступны только внутри платформы, без возможности выгрузки или связки с внешним миром.

Следующий аналог - **T-Bank Invest API**[9]. В рамках рассмотрения существующих решений можно привести множество подобных инструментов от брокеров, например Alfa API, Sber API и т.д., но выбранное решение является одним из самых популярных от брокеров. Представляет собой программный интерфейс для получения информации по биржевым данным. Также у решения имеется официальное Python SDK [10] для взаимодействия с платформой.

Система представляет собой мощный инструмент для получения как данных в реальном времени, так и исторических данных. Поддерживается стандартный для брокерского решения функционал - управление портфелем, выставление заявок на покупку/продажу с определенными условиями.

Ограничения T-Bank Invest API:

- Как и у любых других аналогичных инструментов, имеет привязку к брокеру. Для работы в системе нужен открытый брокерский счет;
- Имеет доступ не ко всем инструментам Московской биржи, т.к. доступны только бумаги, по которым проводятся торги через брокера;
- Как и многие другие решения, является лишь средством получения данных. Нет возможности по выгрузке и хранению данных для дальнейшей работы с ними.

Итогом рассмотрения T-Bank Invest API (и других брокерских решений) является то, что они предназначены в первую очередь для трейдинга, совершения сделок и ведения портфеля, а не для получения и анализа данных в рамках дальнейшего построения процессов.

SmartLab [11] - популярный информационный биржевой портал. Предоставляет возможности для просмотра биржевых данных, таких как акции и котировки, а также имеет новостную ленту с различной биржевой информацией. Порталом пользуется большое сообщество инвесторов и трейдеров, которые могут вести свой блог и аналитику.

Платформа предоставляет обширные социальные возможности. Инвесторы могут вести свой блог и общаться с другими трейдерами, предоставлять и комментировать аналитику на форуме. Также на сайте публикуются общие новости компаний и акций.

Недостатки:

- Не имеет собственного API, из-за чего отсутствует возможность получать какие-либо данные с системы извне;
- Нет возможности выгрузки, хранения данных;
- Зависимость от людей, так как система представляет собой социальное сообщество, где новости и блог пишут люди, контент может содержать неточности.

Выводы по SmartLab можно сделать следующие - полезный информационный ресурс с большим сообществом, но используется исключительно для ручной работы с рынком биржевых

данных, и обсуждения новостей и информации по данной теме, но не предоставляет никаких инструментов для работы с ним.

В заключение главы хотелось бы привести сравнение вышеперечисленных аналогов с предлагаемой в рамках работы системой. Система является комплексным решением, которое реализует полноценный ETL-подход, включающий в себя загрузку данных из источника, преобразование их к внутренней модели, и загрузку в БД, представленной в виде двух хранилищ OLTP и OLAP. Данные сохраняются в БД, вследствие чего доступен любой функционал по работе с ними. В данной системе эта работа представлена в виде построения витрин, и последующих дашбордов, а также взаимодействие с помощью Telegram-бота.

Также, в системе доступен и используется функционал для автоматизации сбора данных. Реализовано это через задания, которые выполняются по расписанию, и имеют встроенные инструменты для мониторинга во время выполнения и отслеживания статусов выполнения. Хранение в разделенных БД даёт возможность как получения исторических данных за промежутки времени (OLAP-хранилище), например для отображения информации на дашборде, так и получение актуальных данных «в моменте» (OLTP-хранилище), например для отслеживания текущих цен на акции. Оба хранилища обладают высокой скоростью доступа к данным, так как данные уже загружены и лежат в БД, а не запрашиваются и преобразовываются каждый раз из API биржи.

При этом разрабатываемая система также имеет ряд недостатков. Однако, архитектура системы позволяет перекрывать эти недостатки путем масштабирования.

- На данный момент, решение работает только с данными с Московской биржи MOEX. Однако, для добавления любого другого источника достаточно реализовать отдельный адаптер для получения данных из него, и маппер для преобразования в модель данных системы. Процессы загрузки в хранилища, и дальнейшей работы с данными меняться не будут;
- Требуется определенных технических навыков для работы с данными в БД, а именно SQL. Однако, это является обратной стороной гибкого хранения данных - из-за возрастающих возможностей по хранению данных, растет и количество кейсов, в рамках которых эти данные можно использовать;
- Требуется определенных технических навыков для развертывания всего сервиса. Из-за того, что сервис состоит из множества сервисов (БД, Airflow, и так далее) на данный момент это необходимо поднимать вручную. Однако, за счет контейнеризации, данный процесс не является критично трудозатратным. Также, в случае размещения всей системы на сервере, конечный пользователь будет работать только с дашбордом или Telegram-ботом.

Итого, в данной главе были рассмотрены аналоги к разрабатываемому решению, приведено их описание и представлены недостатки. Также, аналогично сделано и для предлагаемой системы. По итогам рассмотрения приведена сравнительная таблица 2. Получен результат сравнения: существующие на рынке решения не закрывают полный цикл по работе с данными, ведь являются в большинстве своем лишь частью системы такого цикла. Существуют универсальные ETL-платформы, например Apache NiFi, но построение системы в данных платформах также не решает полной задачи, так как они закрывают только вопрос перемещения данных, в то время как

Таблица 1: Сравнительная таблица существующих решений

Критерий	AlgoPack	Finam	Trading View	T-Bank API(и др.)	Smart Lab	ВКР система
Автоматизация сбора	–	–	–	–	–	+
Трансформация данных	–	–	–	–	–	+
Хранение данных	–	–	–	–	–	+
Исторические данные	+	±	±	±	–	+
Данные в реальном времени	+	±	+	+	±	+
Аналитика	–	–	±	–	–	+
Скорость доступа	–	–	±	–	±	+
Открытый доступ	±	+	±	±	+	+
Независимость от брокера	+	±	+	–	+	+
Несколько источников	–	+	+	±	+	–
Простота начала работы	±	±	+	±	+	–

разрабатываемая система включает в себя также хранение, трансформацию внутри хранилища, аналитику, и последующую работу с данными (дашборд, бот). Разрабатываемая система является комплексным решением, покрывающим полный цикл ETLT-процесса для биржевых данных МОЕХ. Подробное описание API приведено далее в главе 2.6.

2 Проектирование системы

2.1 Формальная постановка задачи

Система на вход принимает данные Московской биржи MOEX двумя способами. Первый - получение по REST API. В основном используется для получения исторических данных с указанием промежутка времени для сбора, но также имеет возможности получения и других требуемых данных, например справочники или информация без указания временного диапазона. Сущности, запрашиваемые из API следующие: финансовые инструменты МосБиржи (instruments), информация по индексам (indices), временные свечи (candles), общие итоги торгов по инструменту за день (daily_aggregates), история изменения по индексам (index_history), информация по дивидендам (dividends). Информация по текущим ценам инструмента (current_prices) и текущим ценам индекса (current_indices) доступна для получения как из API, так и через подписку для обновления в реальном времени через WebSockets - вторым способом получения данных из MOEX.

Полученные данные система должна обрабатывать и сохранять в базы данных по двум типам хранилищ - OLTP (PostgreSQL) для хранения актуальной информации по сущности и OLAP (ClickHouse) для исторических данных. На выходе система предоставляет дашборды для анализа, путем подтягивания данных из БД через витрины, и интерфейс взаимодействия через Telegram-бот и подписки на те или иные события по изменению сущностей.

Функциональные требования:

- Автоматический сбор данных по расписанию и в реальном времени;
- Преобразование сырых данных в модель данных системы;
- Хранение в двух БД с разделением по назначению;
- Контроль качества загруженных данных;
- Трансформация внутри хранилища путем построения витрин;
- Визуализация через дашборды;
- Интерфейс для пользователя через бота.

Нефункциональные требования:

- Идемпотентность - загрузка информации проходит корректно и не создает дубли уже имеющихся данных;
- Контейнеризация - все инструменты для работы поднимаются через Docker;
- Масштабируемость - добавление новых источников для загрузки данных без необходимости переписывания ядра системы.

2.2 Описание предметной области и источников

Московская биржа MOEX - крупнейшая биржа России [2]. На ней представлены несколько видов рынков - фондовый (акции, облигации, паи), срочный (фьючерсы, опционы), валютный, денежный [14]. Представляемая система работает с акциями и индексами.

Все данные на бирже можно получить через определенную структуру, которая также отражается в самой структуре API. Сначала указывается тип рынка (engine), после - секция рынка (market), и в конце - режим торгов (board). Более подробная информация по структуре запросов с примерами в главе 2.6.

Термины предметной области:

- Финансовый инструмент (instrument) - конкретные ценные бумаги, которые торгуются через биржу. Например - акции обычные, привилегированные, облигации, паи и т.д. Каждый инструмент имеет уникальный тикер, например SBER, GAZP и т.д.;
- Биржевой индекс (index) - показатель, отражающий состояние группы инструментов. Главный индекс России - индекс Московской биржи (IMOEX) - включает в себя наиболее ликвидные акции крупнейших и динамично развивающихся российских эмитентов[15]. Индекс не торгуется - он показывает общее состояние рынка;
- Свеча (candle) - стандартный биржевой формат для отслеживания изменения состояния цены за промежуток времени. Содержит в себе 4 цены - цена первой (open) и последней (close) сделки, минимальная (low) и максимальная (high) цена. Также, содержит информацию о количестве проданных и купленных бумаг (volume) и обороте в деньгах (value). OHLCV (по первым буквам ранее перечисленных данных) - общепринятый стандарт в биржевой аналитике;
- Дневной агрегат (daily_aggregates) - итоговый сборник статистики по инструменту по итогу торгов за день;
- Дивиденды (dividends) - выплаты держателям акций компании. Не является обязанностью акционерных обществ перед акционерами. Размер выплаты рассчитывается на 1 акцию;
- Текущие цены (current_prices, current_indices) - последняя зафиксированная биржей цена актива (акции, индекса). Обновляется постоянно, пока активны торги по инструменту(ам) на бирже.

Таблица 2: Таблица сущностей

Сущность	Описание	Источник	Периодичность	БД
instruments	Справочник ценных бумаг	REST	1 раз в день	Postgre, CH
indices	Справочник индексов	REST	1 раз в день	Postgre, CH
candles	Временные свечи	REST	По запросу	CH
daily_aggregates	Итоги по торгам за день	REST	1 раз в день	CH
index_history	История значения индексов	REST	1 раз в день, по запросу	CH
dividends	Дивиденды	REST	По запросу	Postgre, CH
current_prices	Текущие цены инструмента	REST, WS	Постоянно	Postgre
current_indices	Текущие цены индекса	REST, WS	Постоянно	Postgre

2.3 Архитектура решения

Система состоит из нескольких компонентов, отвечающих каждый за свой этап работы с данными. На рисунке 1 представлена архитектура системы.

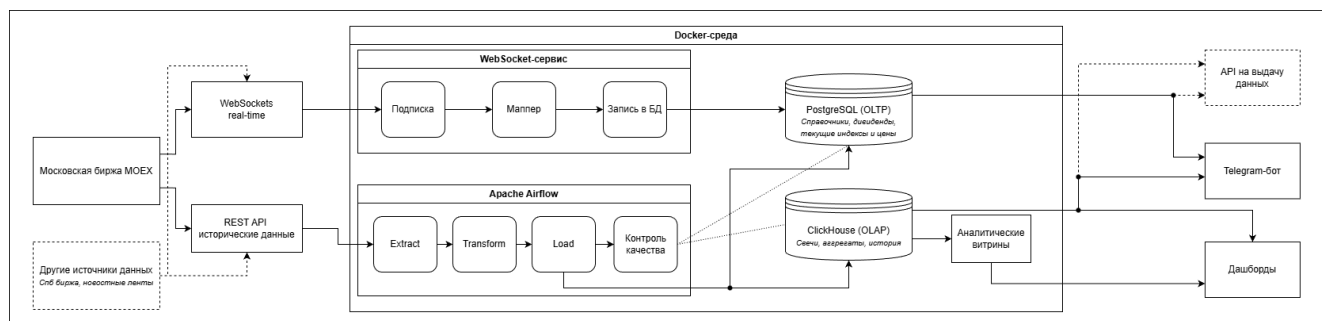


Рис. 1: Архитектура системы

Процесс работы системы проходит слева направо на рисунке. Обработка начинается с источника(ов) данных, происходит получение, преобразование, загрузка данных в хранилища, и предоставление информации пользователю. Далее рассмотрим каждый из слоев системы более подробно.

Источники данных - в предлагаемой системе используется Московская биржа MOEX, но при необходимости на данном этапе можно подключить и другие источники, например СПб биржа, новостные ленты и др.

Получение данных выполняется двумя способами. WebSocket-сервис получает данные по подписке в реальном времени, а развернутый для каждой сущности отдельный ETL-процесс внутри Apache Airflow (с контролем данных в конце) получает исторические данные по REST API. Также, в исторических запросах с большим количеством возвращаемых данных (за теоретически большой промежуток времени) реализовано получение данных по batch-ам, т.к. API не возвращает более 100/500 строк данных.

Сырые данные из MOEX приходят в виде JSON: отдельно массив с столбцами, и отдельно массивы с данными (и др. информация). Для каждой сущности написана своя типизированная модель, в каждой из которых описаны поля сущностей, их типы, значения по умолчанию (при наличии) и требование на обязательность поля. Мапперы преобразуют соответствующие данные из биржи в модели системы.

После преобразования, данные идут в хранилища. Для каждого хранилища написан свой класс работы с данными, который отвечает за загрузку конкретных данных в БД. В PostgreSQL (OLTP) пишутся актуальные данные, а в ClickHouse (OLAP) - исторические.

Разделение OLTP/OLAP обусловлено различающейся необходимостью доступа к данным. Строковая БД (PostgreSQL) оптимальная для быстрого доступа к данным и небольших запросов, поэтому в ней мы храним текущие цены сущностей. Такой вид хранения позволяет отображать изменение информации на дашборде актуальных состояний и в Telegram-боте без задержек. Колоночная БД (ClickHouse) позволяет проводить аналитические операции с большим объемом данных, что позволяет оптимизировать запросы на получение данных за большое время. Поэтому в этой базе данных хранятся исторические данные - свечи, дневные агрегаты, история индексов.

Система реализует не столько классический ETL-подход, сколько ETLT-подход. Первые три шага происходят при выгрузке из API Московской биржи (Extract), маппинге в модели (Transform) и загрузке в базы данных (Load). Четвертый же Transform происходит внутри хранилища ClickHouse путем построения аналитических витрин, которые после используются для дашбордов. Такой подход позволяет более гибко и быстро проводить аналитику с данными, которые уже хранятся в соответствующей БД и имеют унифицированный формат.

Apache Airflow служит оркестратором данных из REST API Московской биржи. Для каждой сущности реализован DAG, который включает в себя выгрузку данных из API MOEX (по batch-ам), преобразование в модели системы, загрузку в БД, контроль качества данных. Эти задачи выполняются по расписанию.

WebSocket-сервис работает отдельно, а не через Airflow, т.к. работает в постоянном режиме, а не является задачей с началом и концом.

Дашборды строятся в Grafana с использованием аналитических витрин из ClickHouse для отображения исторических данных, также производится отображение результатов простых SQL-запросов в PostgreSQL для актуальных состояний.

Telegram-бот считывает актуальные данные из PostgreSQL, что позволяет настраивать алерты на конкретные цены сущностей. Также, с его помощью можно запускать DAG-и Airflow, и получать другую интересующую информацию.

2.4 Обоснование выбора технологий

Выбор технологий происходит, в первую очередь, из-за требований к реализации. Во-первых, необходимо реализовать получение актуальных данных. Как для дашборда, так и для Telegram-бота необходимо быстрое получение информации по одной единственной сущности. Для такой задачи оптимально подходит строковое хранение данных. В качестве СУБД для реализации OLTP подхода был выбран PostgreSQL. С другой стороны, необходимо хранение и агрегации большого количества исторических данных. В таком случае хорошо работают колоночные СУБД, которые как раз специализируются на аналитических запросах. В данной работе был выбран ClickHouse - специализированная OLAP система. Какая-либо одна СУБД не имеет возможности закрыть обе потребности в оптимальном виде. СУБД MongoDB была отвергнута, т.к. является документо-ориентированной, что является избыточным, т.к. данные являются табличными. Также, не под-

держивает SQL. Второй аналог - TimescaleDB. Данная СУБД является расширением PostgreSQL, и не является колоночной как ClickHouse. В теории, конкретно для данной задачи, СУБД могла бы подойти, однако был выбран классический OLTP + OLAP подход, реализуемый двумя разными конкретно направленными СУБД, каждая из которых специализируется на своей части, и делает отдельно это лучше, чем TimescaleDB. Ниже приведена таблица 3 сравнения СУБД.

Таблица 3: Сравнение СУБД по критериям

Критерий	PostgreSQL	ClickHouse	MongoDB	TimescaleDB
ACID-транзакции	+	-	±	+
Колоночное хранение	-	+	-	±
UPSERT	+	±	+	+
SQL	+	+	-	+
Аналитические агрегации	±	+	-	+
Точечные запросы по ключу	+	±	+	+
Партиционирование по дате	±	+	±	+

Каждой сущности необходим свой отдельный процесс выгрузки, преобразования, загрузки в БД, контроля данных, запуск заданий по расписанию. Поддержка проверки состояния процесса, перезапуска в случае неуспеха, логирования. Для всего этого применяются оркестраторы. В данной работе был использован стандартный Apache Airflow. Он предоставляет все нужные возможности - веб-интерфейс с удобным визуалом, отдельные DAG-и, их параметризация. По каждому запуску хранятся логи. Также, из Telegram-бота можно выполнить необходимый запуск с нужными параметрами. У Airflow есть аналоги, такие как Prefect, Dagster, но они слабо распространены и не имеют весомых поводов заменять собой эталонный Apache Airflow.

В системе используются несколько различных сервисов - PostgreSQL, ClickHouse, Airflow, и др. Для простого и удобного поднятия всех этих сервисов используется такой же стандартный и эталонный Docker.

2.5 Модель данных

В этом разделе подробно описаны модели баз данных OLTP и OLAP. В каждом хранилище есть как смысловые по содержанию таблицы, так и справочники.

Схема 2 OLTP хранилища представлена ниже. Основной таблицей является справочник инструментов (instruments). Первичным ключом (PK) является автоинкремент serial поле id. На таблицу sec_types типа инструментов ссылка происходит с помощью внешнего ключа sec_type, отношение один ко многим - один тип ценной бумаги соответствует многим разным инструментам. Также, в таблице есть одно из главных полей - secid, уникальное название инструмента. На это поле ссылаются две другие таблицы: current_prices (текущие цены), отношение один к одному - у каждого инструмента ровно 1 актуальная цена, и corporate_actions (корпоративные события), с отношением один ко многим - множество событий по одному инструменту. Также, есть две отдельные таблицы - справочник индексов (indices), который по secid связывается с текущими ценами индексов (current_indices) с отношением один ко одному по аналогичному текущей цены инструмента принципу.

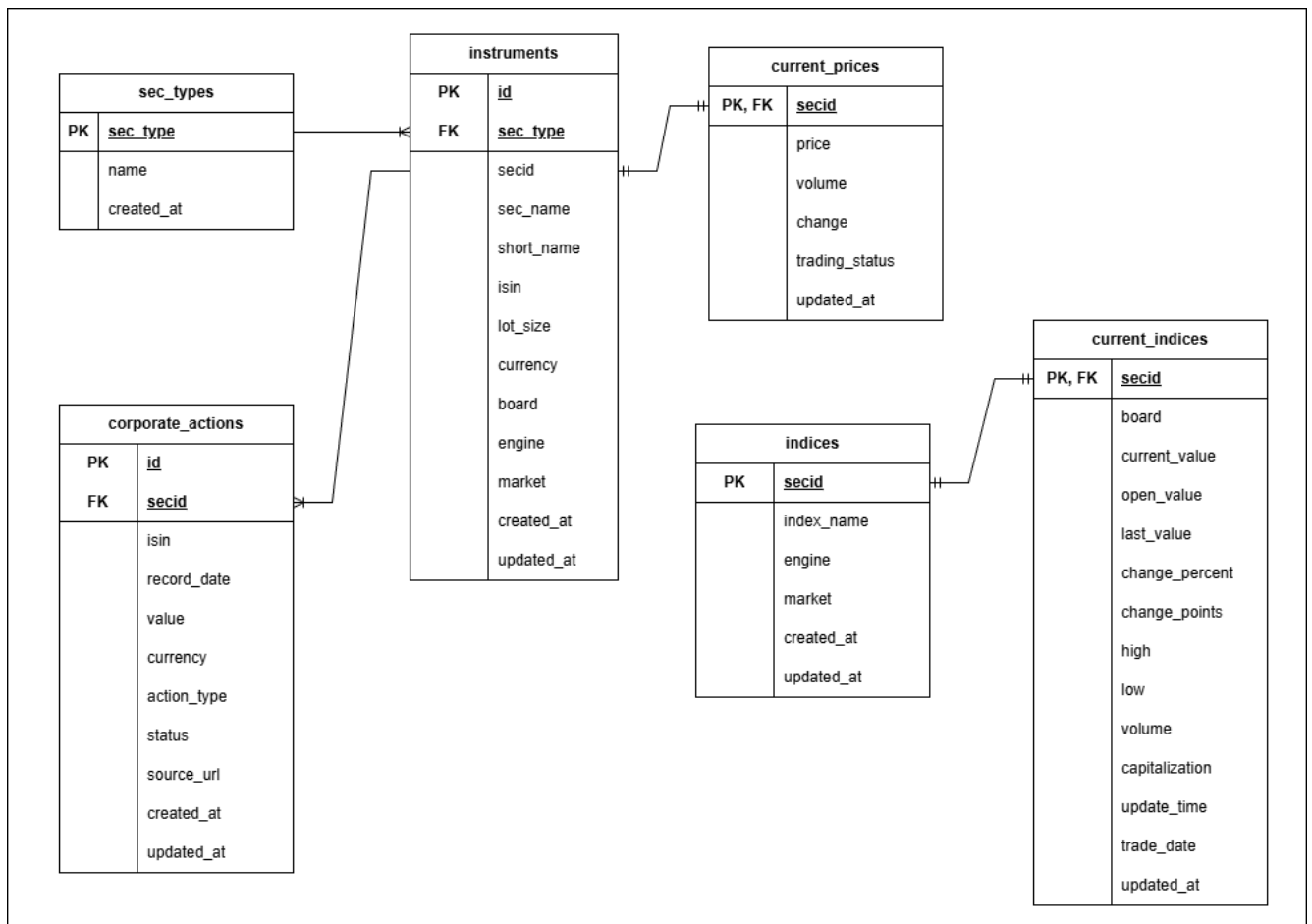


Рис. 2: OLTP модель данных

Справочники `instruments` и `indices` содержат поля `board`, `engine` и `market` - тип, секцию рынка и режим торгов. Эти поля не дублируются в остальных таблицах. Все записи работают через UPSERT, т.е. при дублировании по PK запись не вставляется повторно, а перезаписывает (обновляет) уже имеющуюся. Поля `updated_at` обновляются при каждом изменении того или иного параметра. Для корректного сохранения биржевых денежных данных используется тип `numeric` с фиксированной точностью. Объемы торгов - `bigint`.

Далее рассмотрим схему 3 OLAP хранилища. Основой также служит справочник инструментов `instruments_ref`. По `sec_type` ссылка на таблицу типов инструментов `sec_types`, отношение один ко многим - у множества инструментов могут быть одинаковые типы. На `instruments_ref` по `secid` ссылается 3 таблицы, все отношения один ко многим - на 1 инструмент приходится множество записей. Первая - временные свечи (`candles`), вторая - дневные агрегаты (`daily_aggregates`) и корпоративные события `corporate_actions`. Также, несвязанные с этими таблицы индексов - справочник `indices_ref`, связывается по `secid` с историей индексов (`index_history`) с отношением один ко многим - 1 индекс соответствует множеству исторических изменений.

Все таблицы (кроме `sec_types` за ненадобностью обновления) используют `ReplacingMergeTree` по полю `updated_at`. Это дает аналогичную OLTP возможность при обновлении уже имеющихся по уникальным ключам (заданным в `order by`) данных оставлять только новые, обновляя дату. Также, чтобы не ждать автоматического выполнения дедублирования внутри ClickHouse, в коде всегда

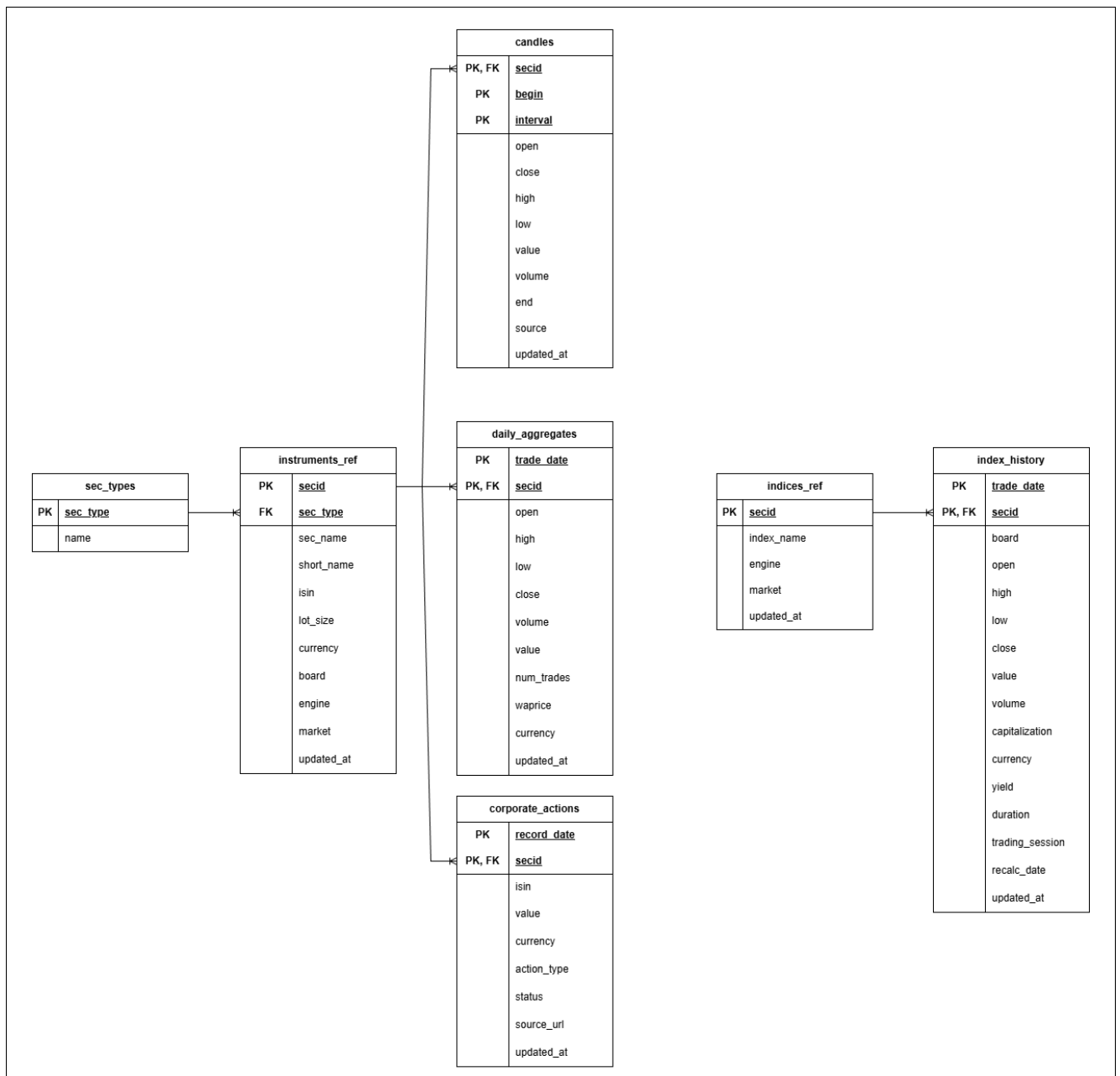


Рис. 3: OLAP модель данных

после загрузки данных выполняется запрос `optimize table ... final`. Таблицы с большими объемами исторических данных (`candles`, `daily_aggregates`, `index_history`) партиционированы по месяцу. Это позволяет быстрее обрабатывать запросу, т.к. СН читает только диапазон, попадающий под требуемое условие запроса. За неимением аналога `numeric` в ClickHouse для денежных данных используется `Float64` - самый близкий по точности тип.

2.6 Описание MOEX ISS API

MOEX ISS (Информационно-статистический сервер) является открытым API для получения биржевых данных. Большая часть классических биржевых данных доступна в бесплатном режиме. Также, есть возможность предоставления информации в режиме реального времени, но с задержкой без подписки[16]. Информацию можно получать в разном удобном для дальнейшей работы виде, например, json или xml.

Ссылки для получения данных из API строятся по иерархии, указанной в 2.2 - в большинстве случаев необходимо указать engine, market и board, а иногда ещё и secid. В конце - эндпоинт, например, securities для инструментов. Ниже представлены все шаблоны URL и конкретные примеры, использующиеся в работе:

- ***<https://iss.moex.com/iss/engines/{engine}/markets/{market}/boards/{board}/securities.json>*** - инструменты по режиму торгов. Примеры:

- *<https://iss.moex.com/iss/engines/stock/markets/shares/boards/TQBR/securities.json>* - получение всех инструментов биржи, а также их текущих цен;
- *<https://iss.moex.com/iss/engines/stock/markets/index/boards/SNDX/securities.json>* - информация по индексам, а также их текущих ценах.

- ***<https://iss.moex.com/iss/engines/{engine}/markets/{market}/securities/{secid}/candles.json>*** - свечи указанного инструмента. Примеры:

- *<https://iss.moex.com/iss/engines/stock/markets/shares/securities/SBER/candles.json>* - получение свечей за все время по SBER. По умолчанию интервал - 10 минут;
- *<https://iss.moex.com/iss/engines/stock/markets/shares/securities/GAZP/candles.json?from=2026-04-01&till=2026-04-15&interval=24&start=0>* - получение свечей по GAZP с параметрами:
 - * from - дата начала выгрузки;
 - * till - дата конца выгрузки;
 - * interval - интервал запроса свечей. В данном случае 24 - день;
 - * start - начиная с какой записи выдавать ответ. Используется для пагинации при выдаче большого количества информации (об этом ниже).

- ***<https://iss.moex.com/iss/securities/{secid}/dividends.json>*** - информация по дивидендам. Пример:

- *<https://iss.moex.com/iss/securities/SBER/dividends.json>* - дивиденды по SBER.

- **`https://iss.moex.com/iss/history/engines/{engine}/markets/{market}/boards/{board}/securities/{secid}.json`** - получение исторической информации: дневные агрегаты, история индексов. Примеры:

- `https://iss.moex.com/iss/history/engines/stock/markets/shares/boards/TQBR/securities/SBER.json?from=2026-03-01&till=2026-04-01&start=0` - дневные агрегаты по SBER, параметры те же;
- `https://iss.moex.com/iss/history/engines/stock/markets/index/boards/SNDX/securities/IMOEX.json?from=2026-03-01&till=2026-04-01&start=0` - история индекса IMOEX за месяц.

Ответы от API в представляемой работе получаем в формате JSON. Ответ содержит несколько секций, например, history - исторические данные, candles - свечи, marketdata - текущие значения, securities - инструменты. В каждой секции находится следующая информация: metadata - типы полей, columns - сами поля, data - данные. Ниже представлено несколько упрощенных примеров ответа:

```
{
  "candles": {
    "metadata": {
      "open": {"type": "double"},
      "close": {"type": "double"},
      "high": {"type": "double"},
      "low": {"type": "double"},
      "value": {"type": "double"},
      "volume": {"type": "double"},
      "begin": {"type": "datetime", "bytes": 19, "max_size": 0},
      "end": {"type": "datetime", "bytes": 19, "max_size": 0}
    },
    "columns": ["open", "close", "high", "low", "value", "volume", "begin", "end"],
    "data": [
      [133.32, 134.95, 135.15, 132.28, 5856073652.600009, 43686280, "2026-04-01 00:00:00", "2026-04-01 23:59:58"],
      [135.12, 133.99, 135.59, 133.21, 4599473714.900002, 34215660, "2026-04-02 00:00:00", "2026-04-02 23:59:59"]
    ]
  }
}
```

Листинг 1: Пример выдачи данных по свечам

```
{
  "dividends": {
    "metadata": {
```

```

    "secid": {"type": "string", "bytes": 36, "max_size": 0},
    "isin": {"type": "string", "bytes": 36, "max_size": 0},
    "registryclosedate": {"type": "date", "bytes": 10, "max_size": 0},
    "value": {"type": "double"},
    "currencyid": {"type": "string", "bytes": 9, "max_size": 0}
  },
  "columns": ["secid", "isin", "registryclosedate", "value", "currencyid"],
  "data": [
    ["SBER", "RU0009029540", "2019-06-13", 16, "RUB"],
    ["SBER", "RU0009029540", "2020-10-05", 18.7, "RUB"],
    ["SBER", "RU0009029540", "2021-05-12", 18.7, "RUB"],
    ["SBER", "RU0009029540", "2023-05-11", 25, "RUB"],
    ["SBER", "RU0009029540", "2024-07-11", 33.3, "RUB"],
    ["SBER", "RU0009029540", "2025-07-18", 34.84, "RUB"]
  ]
}
}

```

Листинг 2: Пример выдачи данных по дивидендам

Как уже упоминалось выше, API не предоставляет в ответе информацию, больше чем 100/500 строк (в зависимости от запроса). Для корректного получения данных в таких случаях для соответствующих методов была написана пагинация, причем в зависимости от ссылки меняется и механизм самой пагинации:

- Для исторических данных API в ответе содержит блок `history.cursor` с тремя полями: `INDEX` - текущая позиция, `TOTAL` - всего строк данных, `PAGESIZE` - размер страницы (100). Для получения всего объема данных здесь используется цикл, который отправляет запросы с параметром `start = INDEX + PAGESIZE`, пока существует следующий набор данных;
- Для свечей цикл также отправляет запросы, пока есть данные, и использует обычное смещение `start = количество уже полученных ранее записей`.

Для получения данных в реальном времени MOEX предоставляет WebSocket API, поверх которого работает протокол STOMP (Simple Text Oriented Messaging Protocol) - текстовый протокол с различными командами для работы с API.

Для авторизации в данной работе используется режим DEMO для демонстрации работы системы. Происходит подписка на инструмент, например: `destination = «MXSE.securities», selector = TICKER=«MXSE.TQBR.SBER»` для акций SBER. После начала работы подписки сервер отправляет все обновления при каждом изменении. Числовые значения приходят в виде [значение, количество знаков], например `'LAST': [316.23, 2]`, `'VOLTODAY': [17183853, 0]`. Пример выдачи данных по подписке:

```

Subscribed: MXSE.securities
{

```

```

'TICKER': 'MXSE.TQBR.SBER', 'CAPTION': 'Sberbank', 'LAST': [316.23, 2], '
CHANGE': [-1.06, 2], 'BID': [316.23, 2], 'OFFER': [316.24, 2], 'HIGH':
[318.08, 2], 'LOW': [315.55, 2], 'HIGHBID': [344.28, 2], 'LOWOFFER':
[300.0, 2], 'BIDDEPTH': 3090781, 'OFFERDEPTH': 4508533, 'VOLTODAY':
[17183853, 0], 'VALTODAY': 5443689277, 'NUMTRADES': 169921, 'LCLOSEPRICE
': [316.24, 2], 'LCURRENTPRICE': [316.21, 2], 'PREVPRICE': [317.29, 2],
'WAPRICE': [316.77, 2], 'PREVWAPRICE': [317.82, 2], '
PRICEMINUSPREVWAPRICE': [-1.59, 2], 'QTY': 1, 'VALUE': [316.23, 2], '
BASEPRICE': None, 'MARKETPRICE': [317.84, 2], 'CURRENCYID': 'SUR', '
SETTLECODE': 'Y1', 'SETTLEDATE1': '2026-04-14', 'SETTLEDATE2': None, '
LOTSIZE': 1, 'FACEUNIT': 'SUR', 'FACEVALUE': [3, 0], 'ISSUESIZE':
21586948000, 'ADMITTEDQUOTE': None, 'TRADINGSTATUS': 'T', 'STATUS': 'A',
'DPVALINDICATORBUY': None, 'DPVALINDICATORSELL': None, 'TIME': '
19:06:49', 'MINSTEP': [0.01, 2], 'PRICEMINSTEP': [0.01, 2]
}
{
'TICKER': 'MXSE.TQBR.SBER', 'BIDDEPTH': 3090773, 'OFFERDEPTH': 4508533, '
VOLTODAY': [17183861, 0], 'VALTODAY': 5443691807, 'NUMTRADES': 169929, '
WAPRICE': [316.77, 2], 'TIME': '19:06:50'
}
{
'TICKER': 'MXSE.TQBR.SBER', 'BIDDEPTH': 3090766, 'OFFERDEPTH': 4508535, '
VOLTODAY': [17183868, 0], 'VALTODAY': 5443694021, 'NUMTRADES': 169936, '
WAPRICE': [316.77, 2], 'TIME': '19:06:51'
}
{
'TICKER': 'MXSE.TQBR.SBER', 'BIDDEPTH': 3090767
}
{
'TICKER': 'MXSE.TQBR.SBER', 'LAST': [316.23, 2], 'CHANGE': [-1.06, 2], '
BIDDEPTH': 3082050, 'OFFERDEPTH': 4509127, 'VOLTODAY': [17183876, 0],
'VALTODAY': 5443696551, 'NUMTRADES': 169944, 'WAPRICE': [316.77, 2], '
PRICEMINUSPREVWAPRICE': [-1.59, 2], 'VALUE': [316.23, 2]
}
.....

```

Листинг 3: Пример выдачи данных по подписке

2.7 Выводы по главе

- Были сформулированы функциональные и нефункциональные требования;
- Описаны сущности, с которыми происходит работа в рамках системы, их источники, БД для хранения, периодичность загрузки;

- Разработана и представлена архитектура системы;
- Приведен разбор выбранных технологий;
- Разработана и представлена модель данных для хранилищ OLTP и OLAP;
- Представлено описание MOEX ISS API, приведены примеры.

3 Реализация

3.1 Адаптеры источников данных

ETLT пайплайн системы состоит из четырех этапов. Первые два - выгрузка (extract) и преобразование (transform) описаны в текущем разделе. В разделе 3.3 - загрузка данных в БД (load), а в 3.5 - вторичная трансформация данных в виде витрин (transform).

Информация из MOEX извлекается двумя способами. Первый - получение исторических и справочных данных через REST API, второй - данные в реальном времени через WebSockets.

REST-клиент в системе реализован с помощью класса *MOEXApiClient*. Класс содержит 8 методов для получения информации по соответствующим сущностям: `get_instruments`, `get_indices`, `get_current_prices`, `get_current_indices`, `get_candles_by_security`, `get_dividends_by_security`, `get_daily_aggregates`, `get_index_history`. Для всех запросов используется библиотека `requests`.

Для простоты управления методами MOEX API, они перенесены в отдельный класс `MOEXUrls`. Также, для вариативности настройки, `engine`, `market`, `board`, `secid` в соответствующих ссылках вынесены как параметры. Примеры: `SECURITIES = f"{BASE_URL}/engines/{engine}/markets/{market}/boards/{board}/securities.json"` или `DIVIDENDS = f"{BASE_URL}/securities/{secid}/dividends.json"`, где `BASE_URL = "https://iss.moex.com/iss"`.

Некоторые методы, а именно `get_index_history`, `get_daily_aggregates` и `get_candles_by_security`, в случае запроса за длительный промежуток времени, возвращают большое количество информации. MOEX API может возвращать только 100 или 500 (в зависимости от метода) строк данных. Первые два запроса возвращают блок `history.cursor`, поэтому принцип пагинации в них используется одинаковый, а именно используя поля `PAGESIZE`, `TOTAL`, `INDEX`. Пример кода:

```
while index + pagesize < total:
    url = MOEXUrls.HISTORY.format(engine=engine, market=market, board=board,
    secid=secid)
    params["start"] = index + pagesize
    data_moex_url = self.send_request(url=url, params=params)

    history_cursor_columns = data_moex_url["history.cursor"]["columns"]
    history_cursor_data = data_moex_url["history.cursor"]["data"][0]

    moex_history_cursors = dict(zip(history_cursor_columns,
    history_cursor_data))
```

```

pagesize = moex_history_cursors.get("PAGESIZE")
total = moex_history_cursors.get("TOTAL")
index = moex_history_cursors.get("INDEX")

data = data_moex_url["history"]["data"]
data_all.extend(data)

```

Листинг 4: Пример пагинации 1

Запрос на получение свечей в методе `get_candles_by_security` не возвращает аналогичного блока, поэтому здесь используется другой принцип, а именно передача параметра `start` равное количеству полученных ранее записей. Код:

```

data_moex_url = self.send_request(url=url, params=params)

columns = data_moex_url["candles"]["columns"]
data_all = data_moex_url["candles"]["data"]

while True:
    params["start"] = len(data_all)
    data_moex_url = self.send_request(url=url, params=params)
    data = data_moex_url["candles"]["data"]
    if not data:
        break
    data_all.extend(data)

```

Листинг 5: Пример пагинации 2

WebSocket подписки реализованы по примеру из официальной документации [18].

Для преобразования данных из MOEX API в модели данных реализованы классы-мапперы `MOEXAPIMapper` и `MOEXWebSocketMapper`. Для каждой сущности написан свой метод, который обрабатывает пустые значения, а также приводит данные в вид модели системы. Разделение мапперов обусловлено как логическим разделением, так и различием в принципе, например, формат числовых данных WebSocket в виде `[value, decimals]`. Пример:

```

def to_current_price(self, moex_data):
    if moex_data.get("LAST") is None:
        return

    return CurrentPrice(
        secid=moex_data.get("SECID").upper(),
        price=moex_data.get("LAST"),
        volume=moex_data.get("VOLTODAY") or 0,
        change=moex_data.get("LASTTOPREVPRICE"),
    )

```

Листинг 6: Пример маппера

Соответственно, для каждой сущности (8 штук) также написана своя Pydantic-модель. Модели описывают форматы данных полей, значения по умолчанию и разрешения полей на пустоту и необходимы для валидации данных и приведения сырой информации в рабочий для системы вид. Пример:

```
class CurrentPrice(BaseModel):
    secid: str
    price: float

    volume: int = 0

    change: Optional[float] = None
    trading_status: Optional[str] = None
```

Листинг 7: Пример Pydantic-модели

3.2 Оркестрация и DAG-и

В системе реализовано 6 DAG-ов, по одному на каждую сущность: `get_candles_dag`, `get_daily_aggregates_dag`, `get_dividends_dag`, `get_index_dag`, `get_index_history_dag`, `get_instruments_dag`. Каждый из них имеет одинаковую структуру - описание настроек через `@dag`, разделение на несколько `@task` по логике выполнения - загрузка информации из MOEX API, а также вставка в БД и проверка качества данных (либо 2 таски, либо 4 в зависимости от того, вставляем мы только в ClickHouse или в CH + PostgreSQL).

Ниже на рисунке 4 представлен пример графа DAG `get_instruments_dag`. На рисунке видно, что задачи по загрузке и проверки двух разных БД работают параллельно. У `get_index_history_dag`, `get_daily_aggregates_dag` и `get_candles_dag` процесс не параллелится и имеет всего одну ветку из трех `@task` в связи с отсутствием записи в PostgreSQL.

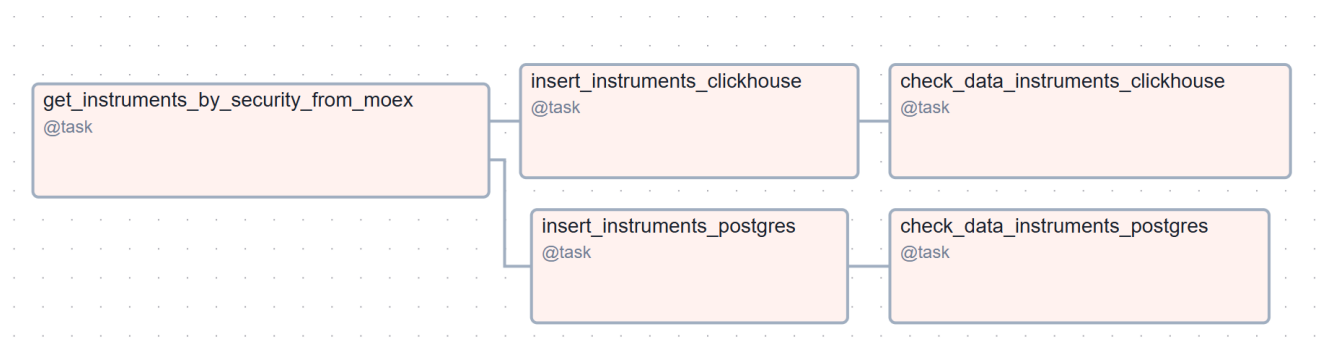


Рис. 4: Граф DAG

Настройки DAG описываются через декоратор `@dag`. Ниже представлен пример для `get_candles_dag`. `start_date` является стартовой точкой начала работы DAG-a. `schedule` задаёт расписание для регулярного запуска DAG и загрузки информации. `catchup=False` отключает загрузку данных за пропущенные промежутки дат, необходимо для того, чтобы в моменте не нагружать как API

Московской биржи, так и саму систему. Последним параметром является params, который включает в себя необходимые для каждой сущности свои параметры, такие как engine, market, board, secid, from, till, interval.

```
@dag(
    start_date=datetime.datetime(2026, 2, 27),
    schedule="@daily",
    catchup=False,
    params={
        "engine": "stock",
        "market": "shares",
        "interval": 24,
        "secid": "SBER",
        "from": "2026-01-01",
        "till": "2026-01-01",
    }
)
```

Листинг 8: Пример настройки DAG

Между задачами данные передаются с помощью механизма XCom Apache Airflow, то есть в каждой задаче присутствует return с данными, которые после передаются в следующую задачу. Pydantic-модели напрямую через XCom не проходят, поэтому используется метод Pydantic model_dump() который превращает сущность модели в словарь Python.

```
def get_candles_by_security_from_moex(**kwargs):
    ...
    return [candle.model_dump() for candle in candles]

def insert_candles_clickhouse(candles_dump, **kwargs):
    ...
    return len(candles)

candles = get_candles_by_security_from_moex()
len_candles = insert_candles_clickhouse(candles)
check_data_candles_clickhouse(len_candles)
```

Листинг 9: Пример XCom

Ниже представлена таблица 4 с настройками каждого DAG.

3.3 Стратегии записи в PostgreSQL и ClickHouse

Для работы с базами данных были написаны 2 DAO класса - PostgresDAO и ClickHouseDAO. Внутри классов реализованы методы для вставки данных в БД для каждой отдельной сущности. Также, при записи данных каждый класс реализует по своему свойство идемпотентности, а имен-

Таблица 4: Настройки DAG

DAG	Источник	БД	Контроль данных	Расписание
get_instruments_dag	REST	PG + CH	наличие, дубли	@daily
get_index_dag	REST	PG + CH	наличие, дубли	@daily
get_index_history_dag	REST	CH	наличие, дубли, заполненность	@daily
get_daily_aggregates_dag	REST	CH	наличие, дубли, заполненность	@daily
get_dividends_dag	REST	PG + CH	наличие, дубли, заполненность	@daily
get_candles_dag	REST	CH	наличие, дубли, заполненность	@daily

но требование, при котором при запуске нескольких раз одних и тех же DAG с одинаковыми параметрами, данные в БД не дублируются.

В PostgreSQL существует стандартный способ для реализации через UPSERT. Ниже приведен пример использования для сущности `instruments`. При получении инструмента с `secid`, который уже есть в таблице, новая запись не создается, а в уже имеющейся актуализируются все параметры, и дата обновления на текущую:

```
INSERT INTO instruments (
    secid, sec_name, sec_type, short_name, isin, lot_size, currency, board,
    engine, market
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (secid) DO
UPDATE SET
    sec_name = EXCLUDED.sec_name,
    sec_type = EXCLUDED.sec_type,
    short_name = EXCLUDED.short_name,
    isin = EXCLUDED.isin,
    lot_size = EXCLUDED.lot_size,
    currency = EXCLUDED.currency,
    board = EXCLUDED.board,
    updated_at = NOW()
```

Листинг 10: UPSERT PostgreSQL

В ClickHouse отсутствует встроенный способ реализации аналогично UPSERT PostgreSQL. Однако, существует другой способ поддержания идемпотентности. В таблицах ClickHouse указывается движок таблицы `ReplacingMergeTree`, и для каждой таблицы задается свой параметр для проверки дубликата, например `ORDER BY (secid, begin, interval)` для таблицы `candles`. Данная настройка позволяет отлавливать дубликаты в таблицах, однако по умолчанию необходимо некоторое время, пока внутри БД автоматически не удалятся дублирования. В условиях постоянно работающей системы с моментальной (по запросу) выдачей данных это неприемлемо, так что в каждой сущности после вставки значений в БД происходит ручной вызов удаления дубликатов таблицы:

```
INSERT INTO moex_olap.daily_aggregates (secid, trade_date, open, high, low,
    close, volume, value, num_trades, waprize, currency) VALUES данные**
```

```
...  
OPTIMIZE TABLE moex_olap.daily_aggregates FINAL
```

Листинг 11: OPTIMIZE TABLE ClickHouse

По умолчанию в ClickHouse стоит ограничение на количество одновременных записей в БД в размере 100 партиций. Таблицы свечей, дневных агрегатов и истории индексов партиционируются по месяцам: *PARTITION BY toYYYYMM(trade_date)*. Соответственно, при загрузке информации за длительный промежуток времени, одна вставка может затрагивать больше 100 партиций, так что в коде при выполнении запросов проставляется параметр *"max_partitions_per_insert_block"*: 0, который убирает это ограничение.

После записи информации в базы данных происходит контроль качества данных, о чем будет рассказано в следующем разделе.

3.4 Контроль качества данных

Контроль качества данных является встроенной в DAG каждой сущности частью, а не является отдельной проверкой со стороны. Это позволяет производить контроль над данными в самом процессе DAG, то есть при непройденной проверке он упадет, выдаст ошибку, будут доступны логи. Внутри каждого DAG после вставки данных в соответствующую БД запускаются таски на контроль этих данных. Реализовано 3 типа проверок: наличие данных, дубликаты, заполненность обязательных полей. Проверки являются общими sql-запросами с параметрами, которые передаются из соответствующих DAG, код реализован отдельно в двух классах для двух БД ClickHouseDataChecker и PostgresDataChecker. Наличие такого контроля данных позволяет гарантировать, что при успешном выполнении запрашиваемые данные присутствуют в БД и готовы к дальнейшей работе.

Контроль наличия данных проверяет существование данных в БД по SQL запросам, соответствующих запросам к MOEX API. То есть, существует 2 вида проверок для обеих БД и 1 только для ClickHouse. Начнем с последней. Она представляет из себя проверку по *secid* за определенные даты, которые передавались в запрос к API для выгрузки. Два других типа проверок используют меньше входных данных. Одна из них - *check_data_exists_by_secid* - запрос по *secid*. Используется для проверки дивидендов, т.к. все дивиденды грузятся разом за всё время по отдельной сущности. Последние проверки - для справочников. Они проверяют по дате обновления *updated_at* за последний день (периодичность обновления справочников). Каждая из проверок наличия данных запрашивает длину ответа в БД, и сравнивает ее с длиной полученных от MOEX API данных. В случае, если количество данных равно - проверка считается пройденной. Ниже приведен пример запроса на проверку

```
def check_data_exists_by_secid_dates(self, table, secid, from_, till_,  
    date_column, data_length):  
    query_check_data_exists_by_secid_dates = f"""  
        SELECT  
            COUNT (*)  
        FROM
```

```

        moex_olap.{table} FINAL
    WHERE
        secid = '{secid}' AND
        {date_column} BETWEEN '{from_}' AND '{till_}';
    """
    db_length = self.clickhouse_dao.client.execute(
        query_check_data_exists_by_secid_dates)[0][0]

    if db_length != data_length:
        raise Exception(f"Несовпадение количества данных. Из API получено {
data_length} записей, в БД {db_length} записей")

```

Листинг 12: Проверка наличия данных СН с датами

Второй вид контроля - дублирование. Является простой проверкой, но особенно важной для ClickHouse, в связи с механизмом работы, описанной в предыдущем разделе 3.3. Проверка происходит через *groupby* в SQL-запросе, причем поля для группировки различаются в зависимости от таблиц. Например, для свечей это *secid*, *begin*, *interval*, а для индексов - только *secid*. Ниже приведен пример такой проверки

```

def check_data_duplicates(self, table, groupby_columns):
    query_check_data_duplicates = f"""
        SELECT
            "{", ".join(groupby_columns)}, COUNT(*)
        FROM
            {table}
        GROUP BY
            "{", ".join(groupby_columns)}
        HAVING
            COUNT(*) > 1
    """
    res = self.postgres_dao.execute(query_check_data_duplicates)

    if res:
        raise Exception(f"Найдены дубли в таблице {table} | {res}")

```

Листинг 13: Проверка на дубликаты

Последняя проверка - на наличие обязательных данных. Эти поля также различаются в зависимости от сущностей, например в свечах необходимо наличие главных полей - *open*, *close*, *high*, *low*, в дивидендах - *value*, а в некоторых сущностях, например справочниках индексов и инструментов, такой проверки вовсе нет. Проверка необходима для отлавливания выдачи пустых записей из API. Код:

```

def check_data_not_empty(self, table, needed_columns):

```

```

not_empty_data_condition = " OR ".join([f"{needed_column} IS NULL" for
    needed_column in needed_columns])
query_check_data_not_empty = f"""
    SELECT
        COUNT(*)
    FROM
        moex_olap.{table} FINAL
    WHERE
        {not_empty_data_condition}
"""
db_empty_count = self.clickhouse_dao.client.execute(
    query_check_data_not_empty)[0][0]

if db_empty_count > 0:
    raise Exception(f"Найдено {db_empty_count} записей с пустыми
        обязательными полями в таблице {table}")

```

Листинг 14: Проверка на наличие обязательных полей

3.5 Витрины данных и дашборды

Ранее, в 2.3. Архитектура решения было описано, что система является больше реализацией ETLT-подхода, а не ETL. Последняя T (Transform) в этом ETLT представляет собой витрины данных. Витрины представляют собой представления (view) внутри БД ClickHouse и используются при построении дашбордов.

В представляемой системе было реализовано 8 витрин: `v_candles_daily`, `v_candles_monthly_agg`, `v_candles_sma`, `v_candles_volatility`, `v_daily_agg_top_tickers`, `v_dividends_yearly`, `v_index_history_capitalization`, `v_index_history_daily`. Ниже приведено более подробное описание каждого представления.

- `v_candles_daily`. Выводит информацию по дневным свечам в формате OHLCV;
- `v_candles_monthly_agg`. Предоставляет агрегированные по месяцам данные дневных свечей - суммарный объем торгов в штуках и оборот в рублях, количество торговых дней;
- `v_candles_sma`. Рассчитывает информацию о скользящем среднем SMA [19] (20, 50, 200), стандартное отклонение от `sma20`, и полосы Боллинджера [19];
- `v_candles_volatility`. Информация о волатильности дневных свечей, которая рассчитывается как отношение разницы максимальной и минимальной цены к цене закрытия;
- `v_daily_agg_top_tickers`. Информация об агрегированном месячном суммарном объеме торгов в штуках, обороте в рублях и количестве сделок. Используется для дальнейшего ранжирования инструментов;

- *v_dividends_yearly* - сумма и количество дивидендных выплат по тикеру по годам;
- *v_index_history_capitalization* - капитализация рынка по дням, не выводит данные с пустой капитализацией;
- *v_index_history_daily* - история изменения цены индекса по дням в формате OHLC + капитализация.

Ниже приведен код вью *v_candles_daily*

```
CREATE OR REPLACE VIEW
    moex_olap.v_candles_daily
AS
SELECT
    secid, toDate(begin) as trade_date, open, high, low, close, volume,
    value
FROM
    moex_olap.candles
WHERE
    interval = 24;
```

Листинг 15: *v_candles_daily*

Для построения дашбордов была выбрана Grafana. Было построено 3 дашборда: Информация по индексу, Информация по тикеру, Текущая информация. Первые два используют витрины из ClickHouse, описанные выше, третий - является сбором общей информации по всем текущим состояниям всех сущностей.

Дашборд «Информация по индексу» представлен ниже на рисунке 5, для примера выбран индекс ИМОЕХ, информация за последние 5 лет. Разделен на несколько смысловых блоков. Сверху - текущая информация - капитализации рынка, значение рынка, последнее изменение в процентах. По середине располагается основной блок - История капитализации и значения индекса за заданное время. Ниже - общие блоки, не связанные с конкретным индексом - топ 10 тикеров за период и сравнение индексов за период.

Второй дашборд - «Информация по тикеру». Представлен ниже на рисунке 6. Для примера выбран *secid* = SBER, также последние 5 лет. Также разделен на несколько блоков. Сверху - техническая информация. Здесь представлена текущая цена инструмента и последнее изменение цены в процентах. Далее - средний объем торгов в штуках и средняя волатильность за последние 30 дней. Последним представлена таблица с информацией о дивидендах по инструменту. Далее идет основной блок. Здесь представлены 4 исторических графика: цена, объем торгов по месяцам, SMA и волатильность.



Рис. 5: Дашборд Информация по индексу

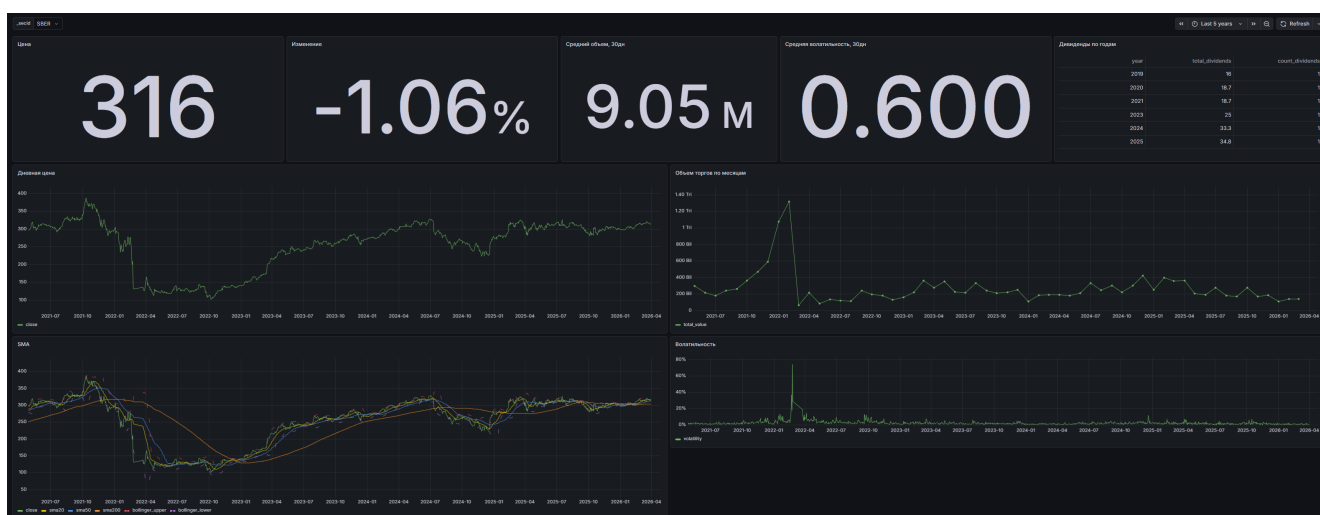


Рис. 6: Дашборд Информация по тикеру

Последний дашборд - «Текущая информация», рисунок 7. Дашборд более простой и не использует витрины, берет информацию только из PostgreSQL. Также разделен по блокам - сверху общая информация - сколько всего исторических записей во всех таблицах в ClickHouse, сколько всего загружено уникальных инструментов и последняя дата обновления информации текущих цен. Далее - блок с текущими ценами. Слева - инструменты - текущая цена, количество проданных+купленных акций за день и изменение в процентах. Справа - индексы - текущее значение, капитализация и изменение в процентах.

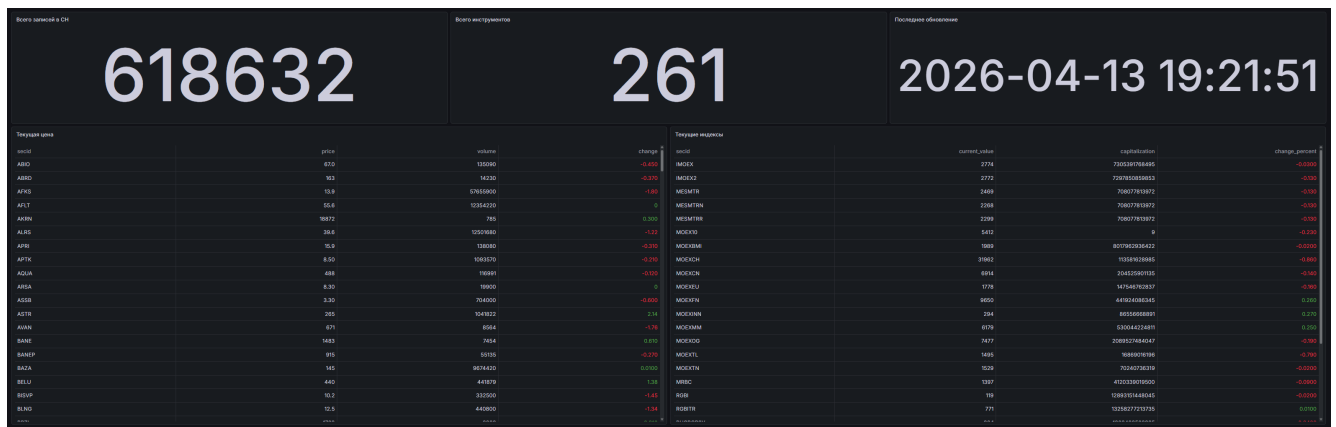


Рис. 7: Дашборд Текущая информация

3.6 Telegram-бот для взаимодействия с системой

Для более гибкой работы с системой был реализован Telegram-бот. В то время как дашборды предоставляют определенные данные, удобные для аналитики, бот предлагает удобный интерфейс для разовых запросов. На данный момент в системе реализована возможность триггера всех DAG Airflow для загрузки данных с передачей всех соответствующих параметров, а также поиск сущностей и получение их текущих цен.

Для реализации бота была выбрана асинхронная библиотека *aiogram*. Код поделен на 3 логические части. Первая - точка входа, *telegram_bot.py*. Вторая - *handlers.py* описывает все доступные команды в самом боте и их логику. Бизнес часть, в нашем случае - работа с БД и Airflow, описана в *services.py*. Ниже представлены все доступные команды бота.

- **/instrument_search name** - поиск secid инструмента по ilike имени. Пример: *instrument_searchСбер*;
- **/index_search name** - поиск secid индекса по ilike имени. Пример: */index_search мосбиржи*;
- **/instrument_current_price secid** - текущая цена инструмента по secid. Пример: */instrument_current_priceSBER*;
- **/index_current_price secid** - текущая цена индекса по secid. Пример: */index_current_priceIMOEX*;
- **/trigger_get_instruments_dag** - запускает DAG по загрузке всех инструментов;
- **/trigger_get_index_dag** - запускает DAG по загрузке всех индексов;
- **/trigger_get_index_history_dag secid from till** - загрузка истории индекса secid с from по till промежуток времени. Пример: */trigger_get_index_history_dagIMOEX2025-01-012026-01-01*;
- **/trigger_get_dividends_dag secid** - загрузка дивидендов по secid инструмента. Пример: */trigger_get_dividends_dagSBER*;

- **/trigger_get_daily_aggregates_dag secid from till** - загрузка дневных агрегатов по secid инструмента с from по till. Пример: */trigger_get_daily_aggregates_dagSBER2025-01-012026-01-01*;
- **/trigger_get_candles_dag secid from till interval** - загрузка свечей по secid инструмента с from по till с интервалом interval. Пример: */trigger_get_candles_dagSBER2025-01-012026-01-0124*.

Взаимодействие с Airflow устроено через отправку POST команды на эндпоинт `/api/v1/dags/dag_id/dagRuns`, с передачей параметров в поле `json` запроса. Ответ приходит в стандартном `requests` виде, после чего в методе хэндлеров проверяется вернувшийся статус, и в зависимости от статуса бот выдает пользователю ответ.

Ниже на рисунке 8 представлен пример некоторых запросов и ответов от бота.

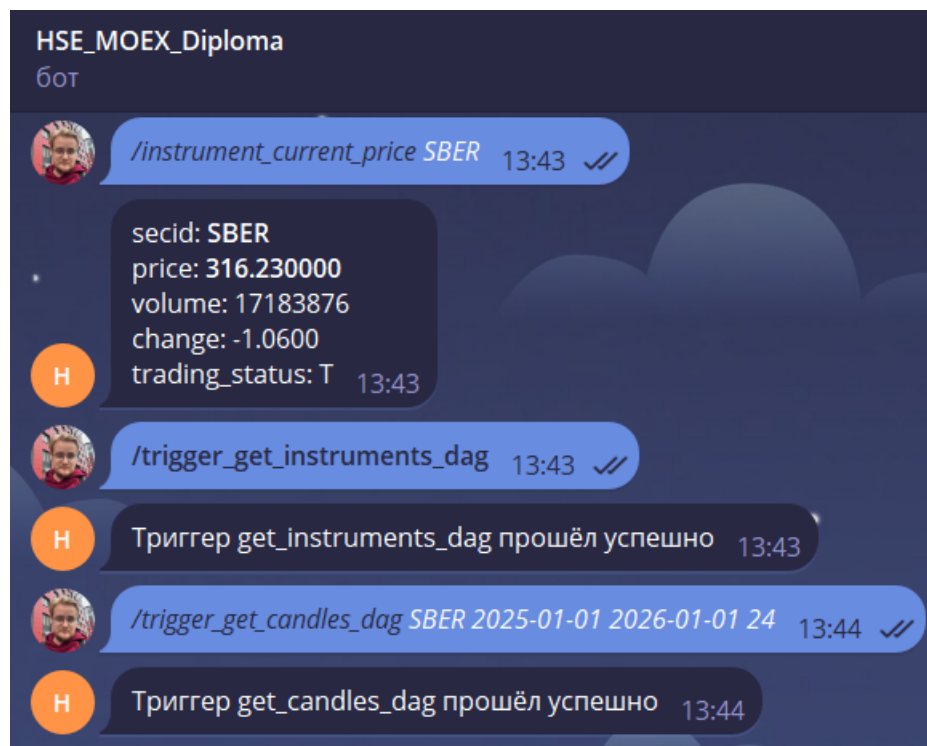


Рис. 8: Telegram-бот

3.7 Инфраструктура и Docker-архитектура

Все компоненты системы обернуты в Docker-контейнеры. Это позволяет упростить развертывание этих компонентов, изолировать сущности друг от друга. Всего отдельно разворачиваются 4 сущности, соответственно, для каждой свой *docker-compose.yml*. Первый блок - это БД (PostgreSQL, ClickHouse) + Grafana, которая разворачивается вместе с базами данных из-за прямого взаимодействия, где БД используются как источники данных для построения дашбордов. Другие 3 блока отдельных Docker-контейнеров: Apache Airflow, Telegram-бот и WebSocket сервис. Всего по итогу система состоит из 8 Docker-контейнеров. Состояния БД и метаданные сохраняются в Docker volumes. Разделенные по логике контейнеры работают в разных сетях, так что их взаимодействие реализовано через DNS адрес *host.docker.internal*, предоставляемый Docker для обращения к хосту. Параметры для настроек и подключений к БД, телеграм-боту и Airflow описаны в *.env* файле.

3.8 Выводы по главе

В главе была представлена реализация системы.

- Разработан ETLT-пайплан загрузки данных из API Московской биржи двумя способами: получение из REST API данных за период времени и справочники, и WebSocket для получения данных в реальном времени;
- Реализована запись отдельно в 2 базы данных: PostgreSQL для хранения актуальной информации по сущностям MOEX и ClickHouse для исторических данных;
- После записи в БД также поддерживается контроль загруженных данных;
- Весь процесс выгрузки из MOEX, преобразования в Pydantic-модели, загрузки в БД и контроль данных обернут в отдельные для каждой сущности DAG-и Apache Airflow;
- По данным из ClickHouse реализованы 8 витрин данных (view), что является последней T (Transform) в ETLT-пайплайне;
- Построены дашборды в Grafana, которые берут данные как из аналитических витрин, так и из PostgreSQL для отображения текущих данных;
- Реализован Telegram-бот для получения информации о текущем состоянии сущностей MOEX API (инструментов и индексов), а также для триггеров DAG Airflow;
- Все компоненты обернуты в Docker-контейнеры.

В качестве будущего развития можно отметить следующие пункты:

- Настройка алертов в Telegram-боте для уведомления при достижении заданной цены инструмента;
- Добавление дополнительных источников данных, кроме MOEX Московской биржи;

- Реализация API на выдачу данных;
- Развитие программной аналитики биржевых данных.

4 Тестирование и экспериментальная оценка

4.1 Функциональное тестирование и эксплуатационные характеристики

В рамках тестирования была проверена работа всех компонентов системы, а именно - получение данных из MOEX API через веб-сокеты и по REST, маппинг данных в модель системы и запись в базы данных, контроль данных после их записи в БД, построение аналитических витрин, отображение данных в дашбордах, работоспособность Telegram-бота. По окончании разработки системы все описанные компоненты работают корректно согласно проектированию. Ниже будут описаны эксплуатационные характеристики тестирования.

Основное функциональное тестирование проводилось на DAG Airflow, т.к. это является основой всего процесса системы и загружает основные данные для будущей работы с ними в БД. Тестирование проводилось в марте-апреле 2026. В таблице 5 ниже приведены результаты тестирования и замеров DAG.

Таблица 5: Тестирование DAG

DAG	Выполнение, сек*	Запусков	Успешные, %	Записей в PG	Записей в CH
get_instruments_dag	7	4	100	261	261
get_index_dag	6	4	100	69	69
get_index_history_dag	15	13	100	-	35935
get_daily_aggregates_dag	10	15	100	-	237759
get_dividends_dag	5	15	100	822	822
get_candles_dag	15	10	100	-	344116**

* - при работе исторических данных время выполнения сильно разнится из-за временных промежутков в запросе. При запросе данных за месяц DAG отрабатывает пару секунд, а, например, запросы за 6 лет работают по 20 секунд и более.

** - для тестирования в рамках 4.2 Эксперимент PostgreSQL vs ClickHouse было загружено всего 4.5 млн данных в таблицу candles, делалось это не через DAG get_candles_dag, а вручную, для большего контроля над загрузкой на таком большом количестве данных.

Помимо DAG, в системе работает WebSocket-сервис для получения данных о текущей цене инструментов или индексов в реальном времени. Функционал работы сервиса был также протестирован - подключение к MOEX, подписка на топик, получение и запись актуальных данных в БД через UPSERT с изменением полей, приходящим в ответе от MOEX и перезаписью даты updated_at на время обновления. Ниже представлены запросы в БД, в которых видны изменения по SBER с измененным updated_at.

```
moex_oltp=# select * from current_prices where secid = 'SBER';
 secid | price      | volume  | change | trading_status | updated_at
-----+-----+-----+-----+-----+-----
 SBER  | 320.580000 | 2118425 | 0.7700 | N               | 2026-05-02 17:02:15.57536
```

```

(1 row)

moex_oltp=# select * from current_prices where secid = 'SBER';
 secid | price | volume | change | trading_status | updated_at
-----+-----+-----+-----+-----+-----
 SBER | 321.710000 | 3668508 | 1.9000 | T | 2026-05-04 06:20:51.299422
(1 row)

moex_oltp=# select * from current_prices where secid = 'SBER';
 secid | price | volume | change | trading_status | updated_at
-----+-----+-----+-----+-----+-----
 SBER | 321.720000 | 3668511 | 1.9100 | T | 2026-05-04 06:20:52.732804
(1 row)

moex_oltp=# select * from current_prices where secid = 'SBER';
 secid | price | volume | change | trading_status | updated_at
-----+-----+-----+-----+-----+-----
 SBER | 321.720000 | 3668544 | 1.9100 | T | 2026-05-04 06:20:58.483642
(1 row)

moex_oltp=# select * from current_prices where secid = 'SBER';
 secid | price | volume | change | trading_status | updated_at
-----+-----+-----+-----+-----+-----
 SBER | 321.710000 | 3668556 | 1.9000 | T | 2026-05-04 06:20:59.199577
(1 row)

moex_oltp=# select * from current_prices where secid = 'SBER';
 secid | price | volume | change | trading_status | updated_at
-----+-----+-----+-----+-----+-----
 SBER | 321.670000 | 3670728 | 1.8600 | T | 2026-05-04 06:21:04.924003
(1 row)

```

Листинг 16: работ WebSocket-сервиса

Также, в рамках функционального тестирования была проведена проверка корректности работы контроля данных, который выполняется в конце каждого DAG после записи данных в БД. За период тестирования все проверки завершились успешно.

4.2 Эксперимент PostgreSQL vs ClickHouse

Как было показано ранее, для хранения данных используется разделение БД OLTP PostgreSQL для хранения актуальной информации по ценам инструментов и индексов и OLAP ClickHouse для хранения исторических данных за промежутки времени. Ниже будут приведены результаты тестирования скорости доступа к данным, на которых и показывается смысл разделения и преимущество ClickHouse для аналитики. Для теста была выбрана таблица candles из СН. Из-за отсутствия аналогичной таблицы в Postgre, она была создана вручную (структура совпадает, индексы созданы). Обе таблицы были наполнены данными свечей из MOEX API, с параметром interval = 1 (минутные свечи) за 2 года. Всего по итогу строк данных в таблицах - 4419373. В качестве запросов

для замеров времени выдачи информации были выбраны следующие: агрегация по одному тикеру, группировка по месяцам по всем тикерам, подсчет количества с условием и статистика по всей таблице. Также, был выбран запрос для поиска одной единственной записи по таблице, на которой Postgre отработывает быстрее. Такой запрос был приведен для чистоты проведенного сравнения аналитических запросов. Ниже приведены запросы в БД ClickHouse (для PostgreSQL - аналогичные, только с explain analyze в начале).

```
1. select avg(close), max(high), min(low), count(*) from moex_olap.candles
   where secid = 'SBER';

2. select secid, toStartOfMonth(begin) as month, avg(close), sum(volume)
   from moex_olap.candles group by secid, month order by secid, month;

3. select secid, count(*) from moex_olap.candles where close > 1000 group
   by secid order by count(*) desc;

4. select count(*), avg(close), avg(volume) from moex_olap.candles;

5. select * from moex_olap.candles where secid = 'SBER' and begin = '
   2026-04-29 19:00:00';
```

Листинг 17: SQL-запросы для замеров времени

Ниже приведена таблица 6 с результатами сравнения. По итогам сравнения видна обусловленность разделения хранения данных по двум БД. Аналитические запросы, работающие на большом количестве данных, выполняющие различные функции, работают быстрее в ClickHouse, поэтому эта БД используется как OLAP-хранилище для исторических данных, и, в дальнейшем, для построения аналитических витрин. Запрос на точечный поиск информации отработывает гораздо быстрее в PostgreSQL, поэтому эта БД используется как OLTP-хранилище для получения точечных информационных по текущим ценам сущностей.

Таблица 6: Сравнение PG vs CH

Запрос	Выполнение PG, мс	Выполнение CH, мс	Запусков	Разница
Агрегация по тикеру	60	19	5	CH > PG в 3 раза
Группировка по месяцам	435	72	5	CH > PG в 6 раз
Подсчет с условием	170	54	5	CH > PG в 3 раза
Общая статистика	220	15	5	CH > PG в 15 раз
Выборка 1 строки	0.03	10	5	CH < PG в 300 раз

4.3 Выводы по главе

- Система прошла функциональное тестирование на реальных данных;
- Приведены результаты тестирования;

- Обусловленность разделения хранения данных в 2 БД показана на практике.

Заключение

В рамках магистрского диплома «Реализация пайплайна загрузки данных» была проведена работа с биржевыми данными. Были исследованы и описаны существующие инструменты для работы с биржевыми данными, рассмотрены их преимущества и недостатки. По итогам рассмотрения было принято решение о необходимости создания комплексной системы, которая покрывает собой полный цикл работы по анализу данных биржи. В рамках разработки системы была построена архитектура такой системы, и реализована сама система. По итогам разработки она включает в себя: два сервиса для получения данных - в реальном времени через WebSockets и исторические через REST API, ETLT процесс по получению, преобразованию и записи в БД информации из биржи, включая контроль данных после записи, логически разделенное хранение данных на 2 БД - OLTP-хранилище PostgreSQL для актуальной информации и OLAP-хранилище ClickHouse для исторических данных. В СН также реализованы аналитические витрины данных - SQL представления (что является последней Т в ETLT). Пользователю данные предоставляются в двух видах - 3 дашборда в Grafana (информация по индексу, информация по тикеру и текущая информация), а также Telegram-бот с возможностью взаимодействия с Airflow для триггера DAG, поиск сущностей в БД и возврат текущего состояния инструментов и индексов.

Список литературы

- [1] Число частных инвесторов на Московской бирже превысило 35 миллионов [Электронный ресурс]. — URL: <https://www.moex.com/n76900> (дата обращения: 08.03.2026).
- [2] Частные инвесторы вложили в ценные бумаги на Московской бирже рекордные 2,5 трлн рублей [Электронный ресурс]. — URL: <https://www.moex.com/n96827> (дата обращения: 08.03.2026).
- [3] ALGOPACK: документация [Электронный ресурс]. — URL: <https://moexalgo.github.io/> (дата обращения: 10.03.2026).
- [4] ALGOPACK — DATASHOP Московской биржи [Электронный ресурс]. — URL: <https://data.moex.com/products/algopack> (дата обращения: 10.03.2026).
- [5] ISS — API MOEX: справочник запросов [Электронный ресурс]. — URL: <https://iss.moex.com/iss/reference/> (дата обращения: 01.03.2026).
- [6] Finam: экспорт котировок [Электронный ресурс]. — URL: <https://www.finam.ru/> (дата обращения: 11.03.2026).
- [7] TradingView [Электронный ресурс]. — URL: <https://www.tradingview.com/> (дата обращения: 12.03.2026).
- [8] Pine Script: документация [Электронный ресурс]. — URL: <https://www.tradingview.com/pine-script-docs/> (дата обращения: 12.03.2026).
- [9] T-Bank Invest API: документация [Электронный ресурс]. — URL: <https://tinkoff.github.io/investAPI/> (дата обращения: 13.03.2026).
- [10] T-Bank Invest Python SDK [Электронный ресурс]. — URL: https://developer.tbank.ru/invest/sdk/python_sdk/faq_python/ (дата обращения: 13.03.2026).
- [11] SmartLab [Электронный ресурс]. — URL: <https://smart-lab.ru/> (дата обращения: 14.03.2026).
- [12] MOEX — «О бирже» [Электронный ресурс]. — URL: <https://www.moex.com/s10> (дата обращения: 05.04.2026).
- [13] Программный интерфейс ISS [Электронный ресурс]. — URL: <https://www.moex.com/ru/documents/6523> (дата обращения: 05.04.2026).
- [14] Рынки Московской Биржи [Электронный ресурс]. — URL: <https://www.moex.com/s4> (дата обращения: 05.04.2026).
- [15] Индекс МосБиржи и Индекс РТС [Электронный ресурс]. — URL: <https://www.moex.com/ru/index/IMOEX/about> (дата обращения: 06.04.2026).

- [16] Программный интерфейс к ИСС [Электронный ресурс]. — URL: <https://www.moex.com/a2193> (дата обращения: 13.04.2026).
- [17] STOMP Protocol Specification, Version 1.2 [Электронный ресурс]. — URL: <https://stomp.github.io/stomp-specification-1.2.html> (дата обращения: 13.04.2026).
- [18] Подключение к ISS+ через Websockets [Электронный ресурс]. — URL: <https://moexalgo.github.io/docs/websocket/websocket> (дата обращения: 18.04.2026).
- [19] Основные индикаторы технического анализа: SMA, EMA и Bollinger Bands [Электронный ресурс]. — URL: <https://tradelink.pro/blog/ru/sma-ema-bollinger-bands-indicators/#простая-скользящая-средняя-sma> (дата обращения: 27.04.2026).

Приложение. Ссылка на репозиторий

https://github.com/AntonDubrovin/HSE_MOEX_Diplom